706. Intersection of Two Sorted Arrays

PLATFORM	DIFFICULTY	DATE
Platform: Custom	EASY	Solved: October 10, 2026

Problem Summary

Given two sorted arrays nums1 and nums2, return their intersection where each element appears min(x, y) times — where x and y are its occurrences in each array.

Approaches

- **Brute Force O(n*m)**: For every element in nums1, scan nums2 for a match. Remove matched element to handle duplicates. Too slow.
- **HashMap O(n+m)**: Count frequencies in both arrays using HashMaps, take min of each key's count. Works for unsorted arrays too.
- **Optimal Two Pointer O(n+m)**: Since both arrays are sorted, use two pointers. Advance smaller pointer, collect on match. O(1) extra space.

Approach

Two pointer technique — since both arrays are sorted, use pointers i and j starting at 0. Compare elements at both pointers — if equal add to result and advance both, else advance the pointer pointing to the smaller value.

Algo

1. Initialize i = 0, j = 0, List lt.
2. While i < nums1.length && j < nums2.length:
3. If nums1[i] == nums2[j] → add to list, i++, j++.
4. If nums1[i] < nums2[j] → i++.
5. If nums1[i] > nums2[j] → j++.
6. Convert list to int[] and return.

Key Insights

- Works only because both arrays are SORTED — smaller pointer advances toward a potential match.
- min(x,y) occurrences handled naturally — both pointers advance together on match.
- Two pointer avoids HashMap — O(1) extra space (excluding output).
- List → array conversion needed since result size is unknown upfront.

Dry Run

i	j	nums1[i]	nums2[j]	action	list
0	0	1	2	1<2 → i++	[]
1	0	2	2	match → i++,j++	[2]
2	1	2	3	2<3 → i++	[2]
3	1	3	3	match → i++,j++	[2,3]
4	2	3	3	match → i++,j++	[2,3,3]
5	3	3	4	3<4 → i++	[2,3,3]

nums1=[1,2,2,3,3,3], nums2=[2,3,3,4,5,7]. Return [2,3,3] ■

Observations

- Last else if can be plain else — only three cases possible (equal, less, greater).
- List to array: loop with index or lt.stream().mapToInt(Integer::intValue).toArray().
- HashMap alternative works for unsorted — two pointer only works when both arrays are sorted.
- Two pointer is optimal for sorted arrays — O(n+m) time O(1) space.

Section

Time Complexity

O(n + m) — each pointer traverses its array at most once.

Space Complexity

O(min(n, m)) — result list holds at most min(n, m) elements.

Points to Remember

- Two pointer on sorted arrays: advance smaller pointer, collect on match — reusable for merge, intersection, union.
- min(x,y) occurrences = both pointers advance on match — duplicates handled automatically.
- HashMap approach works for unsorted — two pointer only works when both arrays are sorted.
- List → int[] conversion: loop with index or stream().mapToInt().toArray().
- This exact pattern is used in: merge sorted arrays (LC 88), union of arrays, 3Sum (LC 15).

Solution Code — Java

```
class Solution {
    public int[] intersectionArray(int[] nums1, int[] nums2) {
        List<Integer> lt = new ArrayList<>();
        int i = 0, j = 0;

        while (i < nums1.length && j < nums2.length) {
            if (nums1[i] == nums2[j]) {
                lt.add(nums1[i]);
                i++;
                j++;
            } else if (nums1[i] < nums2[j]) {
                i++;
            } else {
                j++;
            }
        }

        int[] arr = new int[lt.size()];
        int z = 0;
        for (int n : lt) {
            arr[z++] = n;
        }
        return arr;
    }
}
```